

Ingeniería de Software II

Calidad

Gestión de Calidad

- **Conceptos clave:**
 - La gestión de calidad en software busca **asegurar que los procesos de desarrollo cumplan con estándares definidos** para garantizar la calidad del producto final.
 - Concepto complejo y de aristas múltiples (puntos de vista).
- **Importancia:**
 - **Reduce errores** en etapas tempranas del desarrollo.
 - Mejora la **satisfacción del cliente**.
 - Incrementa la **eficiencia y la sostenibilidad** del software.
- **Métricas de calidad:**
 - Densidad de defectos.
 - Tasa de errores por usuario.
 - Tiempo medio para la resolución de problemas.

LA CALIDAD NO SUCEDE, SE GESTIONA

Calidad – Visiones / Puntos de vista

- El punto de vista trascendental
- El punto de vista del usuario
- El punto de vista del fabricante
- El punto de vista del producto
- El punto de vista basado en el valor

Calidad – Visiones / Puntos de vista

- **El punto de vista trascendental:** es algo que se *reconoce de inmediato*, pero que no es posible definir explícitamente.



Calidad – Visiones / Puntos de vista

- **El punto de vista del usuario:** la calidad en términos de las *metas específicas del usuario* final. Si un producto las satisface, tiene calidad.



Calidad – Visiones / Puntos de vista

- **El punto de vista del fabricante:** en términos de las *especificaciones originales del producto*. Si éste las cumple, tiene calidad.



Calidad – Visiones / Puntos de vista

- **El punto de vista del producto:** la calidad tiene que ver con las características inherentes (*funciones y características*) de un producto.



Calidad – Visiones / Puntos de vista

- **El punto de vista basado en el valor:** de acuerdo con lo que *un cliente está dispuesto a pagar* por un producto.



- **Impacto de las visiones:**
 - Ayudan a definir objetivos claros en las pruebas.
 - Aseguran que el software cumpla con necesidades específicas.

Calidad – Costo de Calidad y "No Calidad"

- **Costo de calidad (CoQ):** Se refiere a los gastos necesarios para prevenir defectos y garantizar un producto de alta calidad.
 - Ejemplo: Entrenamiento, revisiones, auditorías.
- **Costo de "no calidad":** Gastos derivados de errores o fallos en el producto.
 - Fallos internos: Reparaciones previas al lanzamiento.
 - Fallos externos: Reclamos de clientes, pérdidas de reputación.
- **Conclusión:** Invertir en calidad reduce significativamente los costos de "no calidad".

Caso especial de "No Calidad": Deuda técnica

Definición: la deuda técnica es el costo de retrabajo que aparece cuando se prioriza una solución rápida por sobre la más efectiva.

Origen y analogía: el término lo acuñó Cunningham* (1992) y se compara con la deuda financiera: puede ser "buena" si se gestiona, o "mala" si se ignora.

Cuadrantes de Fowler*: prudente + deliberada, imprudente + deliberada, prudente + inadvertida, imprudente + inadvertida; sirven para calibrar intención y nivel de riesgo.

Origen: puede ser funcional o tecnológica

* Howard Cunningham: XP, Manifiesto Ágil, Wiki

* Martin Fowler: *"Refactoring: Improving the Design of Existing Code"*



Deuda técnica



Cuadrante	¿Qué lo caracteriza?	Ejemplo típico en un proyecto real
Prudente + Deliberada ("decisión estratégica")	El equipo sabe que genera deuda y planea cómo y cuándo pagarla.	Sprint de lanzamiento: para llegar a la demo con el cliente, se omite la capa de validaciones complejas y se documenta una "tarjeta" de backlog con el refactor previsto para el próximo ciclo.
Imprudente + Deliberada ("lo hacemos rápido y después vemos")	Se conoce el riesgo pero se acepta sin plan de pago; prima la presión de negocio.	Parche de producción a las 3 a. m.: se mete código hardcodeado con credenciales para reactivar un servicio crítico; "mañana lo limpiamos"... y pasa un año .
Prudente + Inadvertida ("nadie podía preverlo")	Se trabajó con buenas prácticas, pero apareció información nueva .	Cambio de normativa: se construye un módulo fiscal según la ley vigente; tres meses después sale una nueva resolución impositiva que obliga a reescribir gran parte de la lógica.
Imprudente + Inadvertida ("no sabíamos que era tan grave")	Falta de conocimiento o control; la deuda surge sin que el equipo se dé cuenta .	On-boarding apurado: desarrolladores juniors copian-pegan código de internet sin tests ni revisión; meses más tarde aparecen bugs y duplicación masiva de lógica en toda la base de código.

Caso especial de "No Calidad": Análisis de Caso

- Caso: "SGRH 1.0" — sistema de gestión de RR.HH. en PHP que envejeció mal
- Contexto. En 2013, una PyME lanza un sistema en PHP para legajos, licencias y recibos. Se hizo rápido, barato y funcionó años. Nadie actualizó librerías ni runtime.
- Stack original (simplificado).
 - PHP 5.4 con mysql_* (extensión vieja)
 - Framework casero basado en MVC
 - jQuery 1.7 + Bootstrap 2, pegados en /public/vendor/
 - PDF: FPDF 1.x copiada a mano
- Cero tests automatizados; despliegue por FTP
- Señales de envejecimiento.
 - 2017–2019: el hosting fuerza PHP 7 □ cientos de warnings por funciones deprecadas.
 - 2020–2023: proveedor de la lib PDF abandona el repo; fallan documentos con UTF-8.
 - 2024–2025: el hosting anuncia PHP 8.2/8.3 obligatorio □ la app no arranca (errores fatales por mysql_*).
- Nadie quiere tocar el framework casero; la documentación está desactualizada y el dev original ya no está.

Caso especial de "No Calidad": Análisis de Caso

- **Prudente + Deliberada:** "Sabemos que es deuda, la tomamos con control.": se iba a migrar pero el plan nunca se ejecutó
- **Imprudente + Deliberada:** "Sabemos que está mal y aun así lo hacemos.": no migrar para ahorrar tiempo
- **Prudente + Inadvertida:** "No sabíamos que era deuda; parecía razonable.": no era viejo hasta que un día lo fue
- **Imprudente + Inadvertida:** "Ni sabíamos ni medimos el riesgo": no hacemos unit test porque siempre funcionó

Caso especial de "No Calidad": Deuda técnica

Tipos según Steve McConnell (autor de "Code complete"):

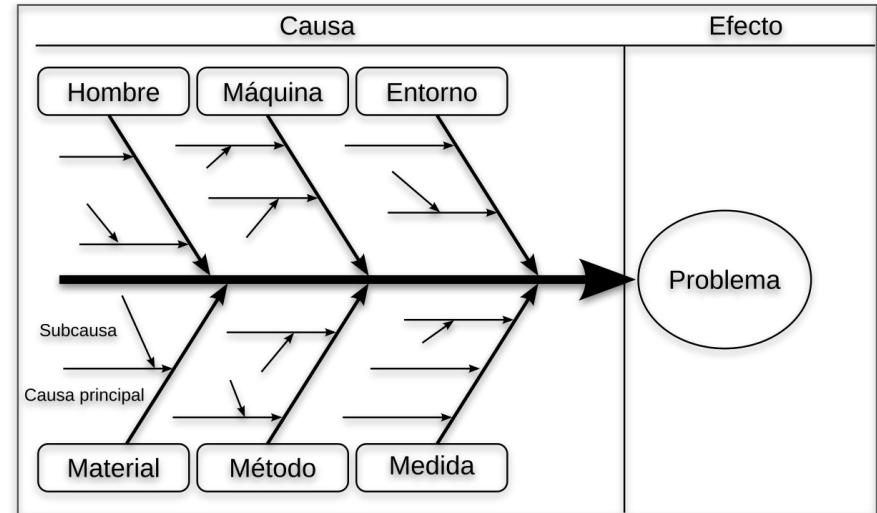
- **Intencionada:** decisión consciente de optimizar el presente (puede ser a corto o largo plazo).
 - Ejemplo: elegir un **framework rápido pero de bajo rendimiento** para llegar a un deadline; requerirá reescritura posterior.
- **No intencionada:** errores, falta de conocimiento o diseño defectuoso.
 - Ejemplo: liberar código escrito por **desarrolladores junior sin revisión suficiente**, generando errores que aparecen tras el lanzamiento.

Buenas prácticas de control:

- **Registrar** cada deuda en un sistema de seguimiento/backlog con esfuerzo y fecha estimada.
- Tratarla como historias de usuario (en enfoques ágiles); si supera 90 días, **marcarla como crítica**.
- Fomentar transparencia y comunicación: **pagar la deuda en incrementos pequeños y visibles** para todo el equipo.

Calidad – Proceso de Gestión de Calidad

- **Fases principales:**
 - Planificación de calidad: incluir en el plan
 - Control de calidad (QC)
 - Aseguramiento de calidad (QA)
- **Herramientas utilizadas:**
 - Diagramas de causa-efecto (Diagrama de Ishikawa o “fishbone”)
 - Auditorías internas.
 - Indicadores clave de rendimiento (KPIs).
- **Ciclo de mejora continua:** Planificar, Hacer, Verificar, Actuar (PDCA).



Calidad – Control de Calidad (QC)

Enfoque

- **Producto** y entregables concretos
- **Detección** de defectos existentes

Actividades

- **Revisiones** de código
- **Pruebas** (unitarias, integración, sistema)
- **Verificación** de requisitos

Resultado

- **Lista de defectos** e incidencias
- **Métricas** (defect density, severidad)

Calidad – Aseguramiento de Calidad (QA)

Enfoque

- **Procesos** y metodologías de desarrollo
- **Prevención** de defectos

Actividades

- Definición de **políticas y estándares** (ISO/IEC 25010, CMMI)
- **Auditorías** de proceso
- Capacitación y mejora continua

Resultado

- **Procesos** documentados y validados
- Confianza en la calidad durante **todo el ciclo de vida**

Calidad – Control vs. Aseguramiento

Perspectiva

- QC: Orientado al producto
- QA: Orientado al proceso

Objetivo

- QC: Detectar defectos
- QA: Prevenir defectos

Momento

- QC: Durante y después del desarrollo
- QA: Desde el inicio y durante todo el ciclo

Responsabilidad

- QC: Equipo de testing/inspección
- QA: Toda la organización (equipo SQA -software quality assurance- facilita)

Calidad – Verificación y Validación

- **Verificación:** Asegura que el **producto** se esté **construyendo correctamente** según los requisitos especificados.
 - Ejemplo: Revisiones de código, inspecciones.
- **Validación:** Comprueba que el **producto** final **cumple** con las **necesidades** del cliente.
 - Ejemplo: Pruebas funcionales, pruebas de usuario.
- **Técnicas comunes:**
 - Revisiones técnicas.
 - Pruebas de integración.
 - Simulaciones.

Calidad – Revisiones y Auditorías

- **Revisiones:** Actividades **informales** para identificar defectos en documentos o productos intermedios.
 - Ejemplo: Revisiones por pares □ code review □ pull request (PR).
- **Auditorías:** Inspecciones **formales** realizadas para garantizar el cumplimiento de estándares.
 - Ejemplo: Auditorías ISO.
- **Beneficios:**
 - Detección temprana de errores.
 - Garantía de cumplimiento normativo.
 - Mejora continua.

Pruebas de software – Clasificación de las Pruebas 1/2

Según el *nivel*:

- **Pruebas unitarias:** Validan componentes individuales.
- **Pruebas de integración:** Aseguran que los módulos funcionan juntos.
- **Pruebas del sistema:** Verifican el software completo.
- **“Smoke test” (*)**: Conjunto reducido de pruebas de sistema
- **Pruebas de aceptación:** Realizadas por el cliente para validar requisitos.

(*) Se la llama así porque es “ver si sale humo” apenas se enciende el sistema: si falla, no vale la pena seguir probando.

Según la *técnica*:

- **Pruebas estáticas:** Evaluación sin ejecutar el código (revisiones, inspecciones).
- **Pruebas dinámicas:** Ejecución activa para detectar defectos.

Pruebas de software – Clasificación de las Pruebas 2/2

Según el *alcance*:

- **Funcionales:** Validan **funcionalidades** específicas.
- **No funcionales:** Evalúan aspectos como **rendimiento, seguridad, usabilidad**, etc.

Según la *automatización*:

- **Manuales:** Realizadas por **testers**.
- **Automatizadas:** Usan **scripts** y **herramientas**.

Pruebas de sistema / No funcionales:

- **Pruebas de recuperación:** recuperarse de fallas y **reanudar** el procesamiento
- **Pruebas de seguridad:** intenta verificar que los mecanismos de **protección** (*pentest*)
- **Pruebas de compatibilidad/despliegue:** varias plataformas y bajo **más de un entorno** de sistema operativo.
- **Pruebas de esfuerzo/rendimiento:** demanda **recursos** en cantidad, frecuencia o volumen anormales
- **Pruebas de usabilidad:** analizan la **experiencia** del usuario.
- **Pruebas de escalabilidad:** comprueban cómo el sistema responde al **crecimiento**.
 - Escalabilidad **vertical:** aumentar los **recursos** de un solo servidor (ej: más CPU, memoria)
 - Escalabilidad **horizontal:** agregar más **servidores** para compartir la carga

Pruebas de software – Técnicas y Estrategias de Pruebas

- **Caja negra:** Basada en especificaciones sin conocer el código.
- **Caja blanca:** Basada en la estructura del código.
- **Técnicas híbridas:** Combinan caja negra y caja blanca.
- **Generación de casos de prueba efectivos:**
 - Identificar escenarios clave.
 - Usar técnicas como partición de equivalencia y análisis de valores límite.
- **Gestión de riesgos:**
 - Priorización según impacto y probabilidad de fallos.

Pruebas de software – Herramientas para Pruebas de Software 1/3

1. Automatización
2. Pruebas de Rendimiento
3. Gestión de Pruebas
4. Simuladores y Entornos

Automatización

- **Herramientas:** Selenium, JUnit, TestNG.
- **Para qué sirve:** Automatizar pruebas repetitivas (funcionales, regresión) para ahorrar tiempo y mejorar la precisión.
- **Lenguajes de programación:**
 - Selenium: Java, Python, C#, JavaScript.
 - JUnit: Java.
 - TestNG: Java.
- **Objetivo:** Garantizar la calidad del software integrando pruebas automatizadas en el desarrollo continuo (CI/CD).

Pruebas de software – Herramientas para Pruebas de Software 2/3

Pruebas de Rendimiento

- **Herramientas:** JMeter, LoadRunner.
- **Para qué sirve:** Evaluar el comportamiento del software bajo diferentes cargas (usuarios concurrentes, volumen de datos).
- **Lenguajes de programación:**
 - **JMeter:** Compatible con cualquier aplicación web; se usa con scripts de BeanShell o Groovy.
 - **LoadRunner:** Compatible con varios lenguajes (C, Java, JavaScript, etc.).
- **Objetivo:** Identificar cuellos de botella, asegurar escalabilidad y mejorar tiempos de respuesta.

Gestión de Pruebas

- **Aplicaciones:** Jenkins o Jira.
- **Para qué sirve:** Documentar, planificar y hacer seguimiento de casos de prueba, ciclos y defectos.
- **Objetivo:** Organizar y centralizar el proceso de pruebas para equipos de QA y desarrollo.

Pruebas de software – Herramientas para Pruebas de Software 3/3

Simuladores y Entornos

- **Herramientas:** Emuladores de dispositivos (Android Studio, iOS Simulator), entornos virtuales (VMware, Docker).
- **Para qué sirve:** Probar aplicaciones en múltiples dispositivos y entornos sin necesidad de hardware físico.
- **Lenguajes de programación:** Depende del entorno; usados para aplicaciones móviles (Kotlin, Swift) o web.
- **Objetivo:** Asegurar compatibilidad y funcionalidad en diferentes configuraciones.

Documentos “““importantes””” (1/3)

Plan de Gestión de Calidad de Software: cómo se asegura y controla la calidad durante todo el ciclo de vida del software. Incluye estrategias, métricas, responsabilidades, y actividades para garantizar que el producto cumpla con los estándares establecidos.

Contenidos:

- **1. Introducción:** Objetivo, Alcance
- **2. Política de Calidad:** Declaración de los principios y objetivos de calidad.
- **3. Roles y responsabilidades relacionados con la calidad:** Gerente de Calidad, Equipo de Desarrollo, Equipo de Pruebas
- **4. Estrategia de Calidad:** Requisitos de Calidad, Métricas de Calidad, Revisiones y Auditorías
- **5. Actividades de Control de Calidad:** Revisiones de Código, Pruebas Automatizadas y Manuales, Validación y Verificación
- **6. Cronograma y Presupuesto:** Desglose de tiempo y costos asignados a la calidad.
- **7. Procedimientos de Aprobación:** Criterios y procesos para la aprobación de entregables de calidad.

Documentos “““importantes””” (2/3)

Plan de Pruebas de Software: enfoque, alcance, recursos y cronograma para realizar las pruebas. Describe los tipos de pruebas, los criterios de aceptación y finalización, el ambiente de pruebas y las herramientas a utilizar.

Contenidos:

- 1. **Introducción:** Objetivo, Alcance.
- 2. **Estrategia de Pruebas:** Tipos de Pruebas y Ambiente de Pruebas.
- 3. **Gestión de Riesgos de Calidad:** Identificación y mitigación de riesgos relacionados con la calidad.
- 4. **Herramientas:** Descripción de las herramientas utilizadas (Postman, Selenium, JIRA, Selenium, SonarQube, etc.).
- 5. **Planificación de Actividades:** Tiempos estimados para cada tipo de prueba.
- 6. **Gestión de Incidentes:** Procedimientos para registrar, rastrear y resolver defectos.
- 7. **Informe de Resultados:** Formato y frecuencia de los informes.

Documentos “““importantes””” (3/3)

Casos de Prueba de Software: pasos específicos para probar una funcionalidad o característica del software. Incluye precondiciones, datos de entrada, resultados esperados y obtenidos, para garantizar que cada componente funcione como se espera. Incluye el registro y trazabilidad de las pruebas.

Objetivo:

- Verificar que **cada requisito de negocio se cumpla** sin errores.
- **Detectar defectos antes de la puesta en producción**, reduciendo costo de corrección.
- Servir como **documentación viva** que facilita mantenimiento y regresión.

Buenas prácticas de redacción:

- **Claridad:** una sola verificación por caso; evita ambigüedad.
- **Trazabilidad:** referencia al ID del requisito o user story.
- **Atomicidad:** los casos no deben depender unos de otros.
- **Datos realistas:** usa entradas representativas (y evita información sensible).
- **Resultados observables:** define salidas medibles (UI, API response, BD, logs).
- **Mantenibilidad:** usa nombres descriptivos y versiona los casos junto al código.

Casos de Prueba – Template

ID: TC-<módulo>-<número secuencial>

Título: <acción + resultado esperado>

Prioridad: Alta | Media | Baja

Precondiciones:

- Estado del sistema
- Datos de prueba necesarios

Pasos:

1. ...
2. ...

Resultado esperado:

- ...

Datos de prueba:

- | | |
|------------|---------------------|
| • Entrada | Valor |
| ----- | ----- |
| usuario | “maria@example.com” |
| contraseña | “P4ssw0rd!” |

Notas/Adjuntos:

- Evidencia (captura, log)
- Vínculo a requisito REQ-123

Casos de Prueba – Sugerencias

Sugerencias:

- Usar Gherkin (*): (Given-When-Then) para **que negocio valide los casos** fácilmente.
- Revisar y depurar los casos en **retrospectivas del sprint**; eliminar duplicados.
- **Medir cobertura** (funcional, de requisitos y de código) como métrica de calidad.

() Lenguaje de marcado utilizado por el enfoque BDD (Behavior Driven Development) para definir escenarios de prueba de una manera clara y comprensible para tanto los desarrolladores como los no técnicos*

Checklist rápido:

- ¿Cada requisito crítico tiene **al menos un caso positivo y uno negativo**?
- ¿Existen pruebas de **límites, nulos y formatos inválidos**?
- ¿Hay escenarios de **rendimiento y concurrencia** básicos?
- ¿Se documentaron **datos, precondiciones y postcondiciones**?
- ¿Las pruebas automatizadas **corren limpio** en el pipeline?

Casos de Prueba – Formato Given-When-Then

¿Qué es?

- Plantilla de Behavior-Driven Development que **conecta requisitos de negocio con pruebas ejecutables**.
- Favorece **lenguaje comprensible** para negocio, QA y desarrollo.

Estructura

*Given (contexto): **estado previo** del sistema y datos iniciales.*

*When (acción): **evento**/disparo que el usuario o sistema realiza.*

*Then (resultado): **salida** observable o cambio esperado.*

Buenas prácticas

- **Una sola acción principal** por escenario.
- **Evitar condicionales:** usar escenarios separados para ramas positivas y negativas.
- Mantener las **frases en tiempo presente y voz activa** (el sujeto realiza la acción).

Casos de Prueba – Formato Given-When-Then

Ejemplo funcional (positivo)

Feature: Autenticación de usuario

Scenario: Inicio de sesión válido

Given la usuaria "María" tiene una cuenta activa

When ingresa usuario y contraseña correctos

Then es redirigida al dashboard y ve el mensaje "Bienvenida, María"

Ejemplo negativo

Feature: Autenticación de usuario

Scenario: Contraseña incorrecta

Given la usuaria "María" tiene una cuenta activa

When ingresa una contraseña inválida

Then se muestra el mensaje "Credenciales inválidas"

And no se crea cookie de sesión

Casos de Prueba – Formato Given-When-Then

¿Dónde puedo usar Given-When-Then?

- **Historias de usuario** → se convierte en los criterios de aceptación que validan el “Definition of Done”.
- **Casos de uso** → cada flujo (principal o alternativo) puede desglosarse en uno o varios escenarios G-W-T.
- **Tareas / subtareas técnicas** → útil para describir comportamiento observable de un endpoint, script o micro-servicio.
- **Requerimientos sin formato estándar** → el template impone estructura y hace la especificación testeable.
- **Reglas de negocio** → explicita condiciones y resultados, evitando ambigüedad entre áreas de negocio y desarrollo.
- **Requerimientos no funcionales medibles** (p. ej. performance, seguridad) → “Given carga de 100 TPS... Then tiempo < 2 s ...”.

¿Y cómo “hacemos” Calidad con IA?

Primero: calidad del proceso asistido por IA.

- Antes QA verificaba trabajo humano.
- Ahora QA también tiene que verificar trabajo generado por IA.
- Nuevo paradigma:
 - ¿Cómo validamos código generado?
 - ¿Cómo auditamos prompts?
 - ¿Cómo garantizamos trazabilidad?
 - ¿Cómo sabemos qué parte hizo un humano y cuál un agente?
 - ¿Cómo prevenimos degradación arquitectónica por generación automática?

¿Y cómo “hacemos” Calidad con IA?

Segundo: deuda técnica acelerada por IA.

- La IA produce código “correcto superficialmente” pero puede:
 - duplicar lógica,
 - romper convenciones,
 - crear dependencias innecesarias,
 - degradar arquitectura,
 - generar código difícil de mantener.
- Mejora productividad local pero puede empeorar calidad sistémica.
- Hay trabajos recientes hablando de “AI-induced technical debt” y “hallucinated architecture drift”.

¿Y cómo “hacemos” Calidad con IA?

Tercero: calidad del pipeline de desarrollo.

- La IA empieza a participar en:
 - refinement,
 - documentación,
 - generación de historias de usuario,
 - code review,
 - estimación (recordar los agentes haciendo Poker Planning),
 - análisis de incidentes,
 - retrospectives,
 - DevOps.

¿Y cómo “hacemos” Calidad con IA?

Cuarto: observabilidad y gobernanza.

- Ingeniería moderna: empiezan a aparecer prácticas como:
 - logging de prompts,
 - versionado de prompts,
 - auditoría de outputs,
 - métricas de aceptación de código IA,
 - trazabilidad de decisiones generadas.

¿El prompt es un artefacto de ingeniería?

¿Y cómo “hacemos” Calidad con IA?

Quinto: calidad de sistemas que contienen IA generativa.

- Un sistema tradicional tiene outputs deterministas.
- Un LLM es estocástico (probabilístico)
- Aparecen dimensiones nuevas:
 - factuality: ¿la respuesta sea verdadera y verificable?
 - toxicidad: ¿contenido ofensivo, discriminatorio, violento?
 - estabilidad semántica: ¿respuestas similares ante preguntas equivalentes?
 - reproducibilidad parcial: ¿resultados suficientemente consistentes?
 - alineación: ¿comportamiento del modelo coincide con objetivos, reglas y valores esperados?
 - robustez: ¿resistencia frente a prompts maliciosos, engañosos?
 - grounding: ¿respuesta apoyada en fuentes y datos disponibles?
 - drift del modelo: ¿hay degradación del comportamiento del modelo?

En otras palabras: el testing tradicional queda corto.